

Sep 14, 22 1:16

main.py

Page 1/4

#!/bin/env python3.8

"""
Michael Bentivegna
Deep Learning Assignment 2
Professor Curro

A word on function choice...

Throughout working on this assignment, numerous activation functions were pondered and implemented. Since a sigmoid function was vital towards keeping the output's estimate between zero and one, it was initially tested as the activation function for each hidden layer. However, sigmoids were not able to properly learn the dataset in the desired timeframe. Relu and elu function were then also looked at but an explosion in weights led to improper estimators (this may have been due to how it interacted with other elements in the code). Lastly, the tanh function was tried for each hidden layer and, ultimately, found good y_hat values in a reasonable number of iterations. This was used in tandem with a sigmoid function at the final layer to produce the displayed output graph.

"""

```
import os
import logging
import matplotlib
import matplotlib.pyplot as plt
import numpy as np
import tensorflow as tf
```

```
from absl import app
from absl import flags
from tqdm import trange
from dataclasses import dataclass, InitVar
```

```
script_path = os.path.dirname(os.path.realpath(__file__))
```

```
font = {"size": 10}
matplotlib.style.use("classic")
matplotlib.rc("font", **font)
```

```
# Define default values for flags
FLAGS = flags.FLAGS
flags.DEFINE_integer("batch_size", 64, "Number of samples in batch")
flags.DEFINE_integer("num_iters", 25000, "Number of SGD iterations")
flags.DEFINE_float("learning_rate", 0.04, "Learning rate / step size for SGD")
flags.DEFINE_integer("random_seed", 31415926535, "Random seed")
flags.DEFINE_float("sigma_noise", 0.2, "Standard deviation of noise random variable")
flags.DEFINE_bool("debug", False, "Set logging level to debug")
```

```
@dataclass
class Data:
    rng: InitVar[np.random.Generator]
    sigma: float
```

```
    def __post_init__(self, rng):
        """
```

```
        Returns the two spirals dataset.
```

```
        """
```

```
        n_points = 500
        self.index = np.arange(n_points * 2)
```

```
        # First spiral creation
```

```
        n = (
            np.sqrt((0.98 * rng.uniform(0, 1, size=(n_points, 1))) + 0.02)
            * (2 * np.pi)
```

Sep 14, 22 1:16

main.py

Page 2/4

```
        * 2
    self.x0 = -np.sin(n) * n + rng.normal(size=(n_points, 1)) * self.sigma
    self.y0 = -np.cos(n) * n + rng.normal(size=(n_points, 1)) * self.sigma
```

```
    # Negate both to get second spiral
```

```
    self.x1 = -self.x0
```

```
    self.y1 = -self.y0
```

```
    # Matrix with x first row, y second row, and classifier the last row
```

```
    # 0 -red
```

```
    # 1 -blue
```

```
    self.combined_data = np.hstack(
```

```
        (
            np.vstack((self.x0, self.x1)),
            np.vstack((self.y0, self.y1)),
            np.vstack(
                (
                    np.zeros([n_points, 1]),
                    np.ones([n_points, 1]),
                )
            ),
        )
```

```
    def get_batch(self, rng, batch_size):
        """
```

```
    Select random subset of examples for training batch
```

```
    """
```

```
        choices = rng.choice(self.index, size=batch_size)
```

```
        return self.combined_data[choices, :]
```

```
class Model(tf.Module):
```

```
    def __init__(self, rng):
        """
```

```
    Initialize weights and bias for each layer
```

```
    """
```

```
        hidden_1 = 200
```

```
        hidden_2 = 100
```

```
        self.w0 = tf.Variable(rng.normal(shape=[2, hidden_1]))
```

```
        self.b0 = tf.Variable(tf.zeros(shape=[1, hidden_1]))
```

```
        self.w1 = tf.Variable(rng.normal(shape=[hidden_1, hidden_2]))
```

```
        self.b1 = tf.Variable(tf.zeros(shape=[1, hidden_2]))
```

```
        self.w2 = tf.Variable(rng.normal(shape=[hidden_2, 1]))
```

```
        self.b2 = tf.Variable(tf.zeros(shape=[1, 1]))
```

```
    def __call__(self, coordinates):
        """
```

```
    Operations to execute one iteration of the fully connected neural network
```

```
    """
```

```
        coord = coordinates.astype("float32")
```

```
        out0 = tf.math.tanh(coord @ self.w0 + self.b0)
```

```
        out1 = tf.math.tanh(out0 @ self.w1 + self.b1)
```

```
        predicted_out = tf.math.sigmoid(out1 @ self.w2 + self.b2)
```

```
        return tf.squeeze(predicted_out)
```

Sep 14, 22 1:16

main.py

Page 3/4

```

def main(a):
    logging.basicConfig()

    if FLAGS.debug:
        logging.getLogger().setLevel(logging.DEBUG)

    # Safe np and tf PRNG
    seed_sequence = np.random.SeedSequence(FLAGS.random_seed)
    np_seed, tf_seed = seed_sequence.spawn(2)
    np_rng = np.random.default_rng(np_seed)
    tf_rng = tf.random.Generator.from_seed(tf_seed.entropy)

    # Get sample data
    data = Data(
        np_rng,
        FLAGS.sigma_noise,
    )

    # Initialize model
    model = Model(tf_rng)
    optimizer = tf.optimizers.SGD(learning_rate=FLAGS.learning_rate)

    # Use stochastic gradient descent for training during each iteration
    bar = trange(FLAGS.num_iters)

    for i in bar:
        with tf.GradientTape() as tape:
            output = data.get_batch(np_rng, FLAGS.batch_size)
            y_hat = model(output[:, :2])
            loss = tf.reduce_mean(
                -1 * output[:, 2] * tf.math.log(y_hat)
                - (1 - output[:, 2]) * tf.math.log(1 - y_hat)
            ) + 0.01 * tf.reduce_mean(
                [tf.nn.l2_loss(v) for v in model.trainable_variables]
            )

            grads = tape.gradient(loss, model.trainable_variables)
            optimizer.apply_gradients(
                [
                    (grad, var)
                    for (grad, var) in zip(grads, model.trainable_variables)
                    if grad is not None
                ]
            )
            bar.set_description(f"Loss @ {i} => {loss.numpy():0.6f}")

    # Get probability for each point on the meshgrid
    x = y = np.linspace(-15, 15, 1000)
    x_mesh, y_mesh = np.meshgrid(x, y)
    cont = np.array(list(zip(x_mesh.flatten(), y_mesh.flatten()))))
    out = model(cont).numpy()

    # Plotting spiral data and model estimator
    fig, ax = plt.subplots(1, 1, figsize=(6, 6), dpi=200)

    ax.set_title("Spirals Data & Model Estimate")
    ax.set_xlabel("x1")
    ax.set_ylim(-15, 15)
    h = ax.set_ylabel("x2", labelpad=10)
    h.set_rotation(0)
    ax.contourf(

```

Sep 14, 22 1:16

main.py

Page 4/4

```

        x_mesh,
        y_mesh,
        out.reshape(x_mesh.shape),
        levels=[0, 0.5, 1],
        colors=["#ff6666", "#6eb7cd"],
    )
    ax.plot(data.x0, data.y0, "ro", data.x1, data.y1, "bo")
    plt.savefig(f"{script_path}/fit.pdf")

if __name__ == "__main__":
    app.run(main)

```